

Meal Logging Frontend Implementation Guide

The AI chatbot workflow, every API call to make, and every branch the UI has to handle.

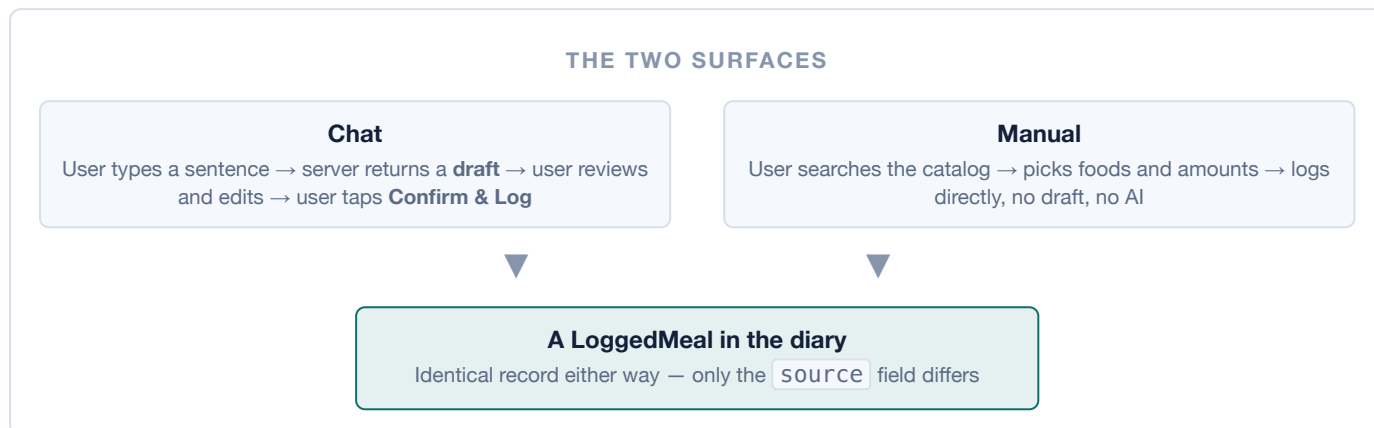
Who this is for. The engineer building the meal-logging and AI chat screens. It assumes you have the base URL and a token, and nothing else. Everything here is derived from the live serializers, so the field names and shapes are what the server actually sends — but generate your types from `/api/schema` rather than transcribing this document.

Base path `/api/v1` · **Auth** `Authorization: Bearer <token>`

Companions [High-Level Overview](#) · [Service Deep Dive](#) · [API Verification Runbook](#)

1 What you are building

Two ways into the diary, one destination. The chat surface is the interesting one; the manual surface is its fallback and its safety net.



The one thing to internalise

THE DRAFT LIVES ON THE SERVER, NOT IN YOUR STATE

A chat message never logs a meal. It creates or mutates a **draft** — a server-owned object you render. You do not build it, merge into it, or recompute its numbers. Every mutation is a round trip that returns the *whole* updated draft, and you replace your local copy with what comes back. The only client-side maths in the entire feature is the portion slider (§7), and even that is persisted by asking the server.

Concretely, that means three rules:

- **Never compute nutrition yourself** except for the live slider preview. The server recomputes at confirm time and its number wins.
- **Always carry `version`** on mutating calls. It is how the server detects that another device changed the draft under you.
- **Replace, don't merge.** Every draft-mutating response returns the full draft. Swap it in wholesale.

2 Setup

Headers on every request

```
Authorization: Bearer <token> // guest token OR Clerk session JWT – same header
Content-Type: application/json
X-Request-Id: <uuid> // optional but send it: it comes back on the response
// and lands in server logs, which makes bug reports traceable
```

You never declare which kind of token you hold — the server routes on the JWT's signing algorithm. Get a guest token from `POST /auth/guest`; once the user signs in with Clerk, send the Clerk session JWT instead.

THE ONE AUTH BEHAVIOUR THAT WILL SURPRISE YOU

After a successful `POST /auth/claim`, the **guest token stops working and returns 401**. That is not an error to retry or a session expiry — it is the signal that the guest's data now lives on the signed-in account. Handle 401 on a guest token by switching to the Clerk session, not by re-authenticating.

The error envelope — one shape for everything

```
{ "error": {
  "code": "draft_version_conflict", // ← branch on THIS, never on the status code
  "message": "...", // human-readable; safe to show in a toast
  "details": { ... }, // varies by code – often carries what you need to recover
  "requestId": "..." // matches the X-Request-Id header; put it in bug reports
} }
```

WHY CODE AND NOT THE STATUS

Several distinct outcomes share a status on purpose. `profile_required` and `plan_required` are both 409 and route to different screens; `open_draft_exists`, `draft_version_conflict` and `draft_not_open` are all 409 and need three different recoveries. Switching on `error.code` is the only correct implementation.

Generate your types — do not hand-write them

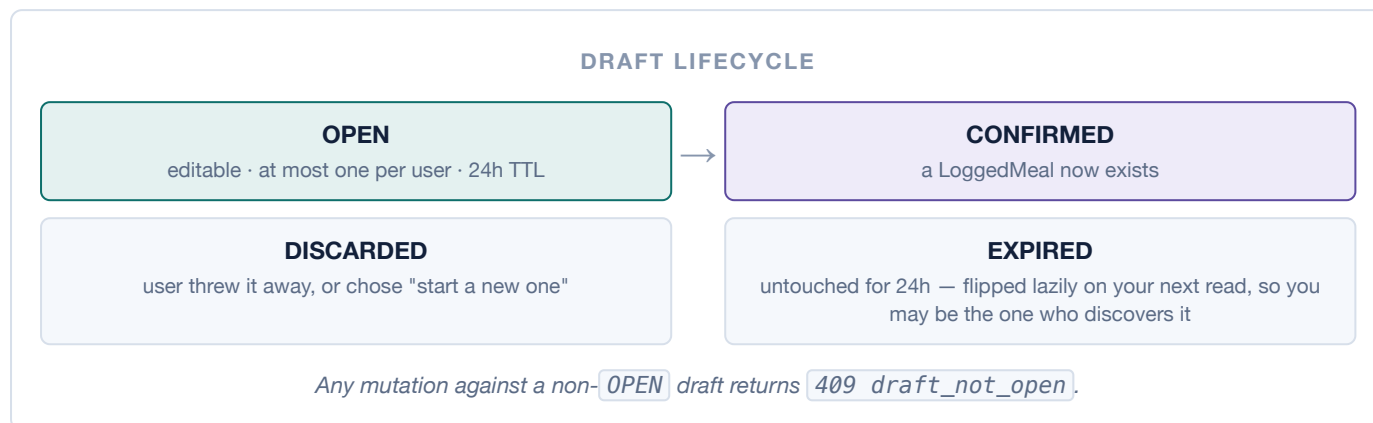
```
# backend, once per contract change
scripts/export_openapi_schema.sh schema.yaml

# frontend
npx openapi-typescript schema.yaml -o src/api/schema.d.ts
```

The schema is generated from the serializers, so it cannot drift from what the endpoints return. Re-run both steps whenever a response shape changes. Swagger UI is at `/api/schema/docs` if you want to poke at it by hand.

3 The draft state machine

Four states, and the rules that constrain what your UI may do in each.



RULE	WHAT IT MEANS FOR YOUR UI
One open draft per user	Creating a second returns <code>409 open_draft_exists</code> , with the existing draft in <code>error.details.draft</code> . Render that instead of erroring — the user almost certainly wants to see it.
Every mutation bumps version	Send the version you last saw. A mismatch returns <code>409 draft_version_conflict</code> with the <i>current</i> draft in <code>error.details.draft</code> — swap it in and re-render. Do not refetch.
Numbers are recomputed on every read	Drafts store no nutrition. Do not cache a draft's totals across sessions; they can legitimately change if the catalog was edited.
Expiry is lazy	There is no background job. A stale draft flips to <code>EXPIRED</code> the moment you read it, so handle a <code>GET</code> that comes back non- <code>OPEN</code> .

GAP YOU MUST DESIGN AROUND: THERE IS NO "LIST MY DRAFTS" ENDPOINT

`POST /assistant/drafts` creates; `GET /assistant/drafts/{id}` reads one. There is **no** `GET /assistant/drafts` that returns the user's open draft. So after an app restart you cannot ask "do I have a draft in progress?" directly.

Two ways to handle it: persist `draftId` locally (AsyncStorage / MMKV) alongside the user id and `GET` it on launch — or recover it from the conflict: attempt whatever the user is doing, and if you get `409 open_draft_exists`, read the draft out of `error.details.draft`. The second works with no local state at all, but only fires when the user tries to start something. Persisting the id is the better default; the conflict path is your fallback when local storage is empty or stale. Worth raising with backend if resume-after-kill matters for launch.

4 Workflow — the chat screen

One endpoint drives the entire conversation. Everything below is a branch on its response body.

A SINGLE CHAT TURN

1 • User types and hits send

Optimistically append their bubble. Generate a

`clientId` (UUID v4) **once** and reuse it for every retry of this same message.

Why the id matters

A retried send with the same id replays the original response instead of parsing twice or mutating the draft twice.

2 • POST /assistant/messages

{ `clientId`, `content`, `onOpenDraft?` }. Show a typing indicator — a T2 call can take a few seconds; the free tiers are instant.

Always HTTP 200

A refusal, a clarification and a created draft are all valid turns. Only 400 (malformed) and 409 (`idempotency_key_reused`) are real errors.

3 • Branch on the body

In this order: `needsClarification` → `draft` → `assistantText` alone. See the decision table in §5.

4 • Render the assistant bubble, then the draft card

Always show `assistantText`. If `draft` is non-null, render or replace the draft card beneath it.

Also surface

`unconsumedText` as a quiet "I didn't catch: ..." line, and the quota pill if `quotaExceeded`.

5 • User edits — by tapping or by typing

Both routes end at the same draft. Tapping uses the structured endpoints (§7); typing is just another message ("make the rice 250g").

6 • Confirm & Log

POST /assistant/drafts/{id}/confirm with an idempotency key. Returns the logged meal *and* today's running totals, so Home updates without a refresh.

5 The message API — request and response

The request

```
POST /api/v1/assistant/messages

{
  "clientId": "3f2a...", // required. UUID v4, generated per message, reused on retry
  "content": "200g rice and 2 pieces roti", // required, max 2000 chars
  "onOpenDraft": "ADD" // optional — ONLY when answering an open_draft clarification
}
```

DO NOT SEND `ONOPENDRAFT` SPECULATIVELY

Send it only when you are replaying a message the server asked you to disambiguate. Sending `"NEW"` by default silently discards whatever draft the user had in progress.

The response — every field, and what to do with it

```
{
  "requestId": "...",
  "messageId": "...",
  "tier": "PARSER", // MANUAL|PRECLASSIFIER|CACHE|PARSER|LLM_SMALL|LLM_LARGE
  "intent": "LOG_NEW", // 13-value taxonomy — see below
  "assistantText": "Got it — let me break that down.",
  "draft": { ... } | null,
  "unconsumedText": ["a bit of something else"],
  "needsClarification": { "reason": "open_draft", "candidates": ["ADD","NEW"] } | null,
  "quotaExceeded": false,
  "gamificationSuppressed": false,
  "wellbeingResources": [ ... ] | null
}
```

FIELD	PRIORITY	WHAT THE UI DOES
<code>needsClarification</code>	Check first	Non-null means nothing was mutated . Render <code>assistantText</code> plus a chip per <code>candidates</code> entry. Do not touch the draft.
<code>draft</code>	Check second	Non-null → render or replace the draft card. Non-null <i>alongside</i> a clarification means "here is the draft you already have", not "here is a change".
<code>assistantText</code>	Always	Render as the assistant's chat bubble. Never suppress it — for non-logging intents it is the entire response.
<code>unconsumedText</code>	Always	Non-empty → show a subdued "I didn't catch: ..." line with a tap-to-search affordance. Never drop it silently; that is the whole reason the field exists.
<code>quotaExceeded</code>	Always	<code>true</code> → the AI escalation was blocked. Show the limit sheet and call <code>GET /assistant/quota</code> for the numbers. Logging still works — say so, and point at quantified phrasing or manual entry.
<code>intent</code>	Conditional	Only <code>WELLBEING_FLAG</code> changes rendering (§9). Everything else is analytics.
<code>tier</code>	Debug	Which layer answered. Useful in a debug overlay and in analytics; do not surface it to users . If you want an "AI" badge, key it off <code>LLM_SMALL/LLM_LARGE</code> .

FIELD	PRIORITY	WHAT THE UI DOES
gamificationSuppressed	Conditional	Once true for this session, suppress streak/deficit-praise copy. Nothing reads it yet — wire it when gamification lands.
requestId · messageId	Always	Store both against the message in your chat list. <code>requestId</code> is what support needs to find the server log.

Intents you will actually see

INTENT	DRAFT?	RENDERING
LOG_NEW	Yes — created	The main path. Assistant bubble + draft card.
EDIT_ITEM · ADD_ITEM · REMOVE_ITEM · SET_SLOT	Yes — updated	Replace the card. Consider animating the changed row.
DIARY_QUERY	No	A real answer with real numbers ("So far today: 1,240 kcal — 620 left toward your 1,860 goal"). Trend questions deep-link to Insights — make that text tappable.
NUTRITION_QA	No	Per-100g facts about one food. Consider linking to that food's detail screen.
APP_HELP	No	Scripted copy pointing at Settings / Help Center. Link those destinations.
ADVICE_SEEKING	No	An explicit decline. Render plainly; do not dress it up.
SOCIAL · UNCLEAR · OTHER	No	Short scripted reply. Nothing special.
WELLBEING_FLAG	No	Special handling — see §9.

Handling the two clarifications

<code>reason: "open_draft"</code> <code>candidates: ["ADD", "NEW"]</code>. Render two chips: "Add to this meal" / "Start a new one". On tap, re-send the same content with a new <code>clientId</code> and <code>onOpenDraft</code> set to the chosen value.	<code>reason: "ambiguous_target"</code> <code>candidates</code> are item names from the draft. Render them as chips; on tap, re-send a more specific phrasing (e.g. "remove the cooked white rice"). Nothing was mutated, so there is nothing to undo.
---	--

WHY A NEW `CLIENTMESSAGEID` ON THE RETRY

The clarification reply is a genuinely *different* request (it carries `onOpenDraft`). Reusing the original id with different content returns `409 idempotency_key_reused`.

6 Rendering the draft card

Three item states, three different cards. Getting this wrong is the most visible way to misrepresent the data.

```
{
  "id": "...", "version": 3, "status": "OPEN",
  "name": "Lunch – Whole Wheat Roti",
  "slot": "LUNCH",
  "parseTier": "PARSER", "confidence": 1.0,
  "totals": { "caloriesKcal": 498, "proteinG": 14, "carbsG": 101, "fatG": 5, "fiberG": 7 },
  "items": [ ... ]
}
```

MACROS ARE NULLABLE – RENDER “–”, NEVER 0

`proteinG`, `carbsG`, `fatG` and `fiberG` are `null` when genuinely unknown (a draft made entirely of estimated dishes). A `?? 0` in your render turns “we don't know” into “it contains none”, which is a real correctness bug. Only `caloriesKcal` is guaranteed non-null.

Item state 1 — **RESOLVED**

```
{
  "id": "...", "resolution": "RESOLVED", "isEstimatedDish": false,
  "foodId": "...", "foodName": "Cooked White Rice",
  "rawText": "200g rice",
  "quantity": 200, "unit": "g", "grams": 200, "state": "COOKED",
  "defaultGrams": 200, // slider baseline – fixed at creation, never moves
  "quantitySource": "EXPLICIT", // EXPLICIT | ASSUMED
  "massSource": "DIRECT", //
  DIRECT|HOUSEHOLD_TABLE|USER_HISTORY|CATALOG_SERVING|CATEGORY_FALLBACK
  "matchBand": "HIGH", // HIGH | MEDIUM | LOW | null
  "caloriesKcal": 260, "proteinG": 5, "carbsG": 56, "fatG": 1, "fiberG": 1,
  "perGram": { "kcal": 1.3, "proteinG": 0.027, "carbsG": 0.282, "fatG": 0.003 }
}
```

SIGNAL	WHAT TO SHOW
<code>quantitySource === "ASSUMED"</code>	An "est." chip on the amount. The user never stated it — the server inferred it from their history or a catalog default. This is the honesty affordance; do not hide it.
<code>massSource === "USER_HISTORY"</code>	Optional but nice: <i>"your usual"</i> . It means the system used this user's own median portion.
<code>matchBand === "MEDIUM"</code>	A plausible but not certain food match. Consider a subtle "tap to change" affordance rather than a warning.
<code>matchBand === null</code>	Either an exact pick (manual entry) or a component of a composite dish. No badge.
<code>rawText</code>	What the user actually said — "Cooked White Rice (Chicken Biryani)" for a dish component. Useful as a secondary line so the expansion is legible.

Item state 2 — UNRESOLVED

```
{ "resolution": "UNRESOLVED", "rawText": "150g quinoa",  
  "foodId": null, "foodName": null, "caloriesKcal": null, "perGram": null, ... }
```

RENDER

The `rawText`, no numbers, and a clear "we couldn't find this" state with a **Search for it** CTA that opens catalog search prefilled. It contributes nothing to the totals, and it will be **silently dropped at confirm** — so make its presence obvious before the user taps Confirm.

Item state 3 — ESTIMATED_DISH

```
{ "resolution": "ESTIMATED_DISH", "isEstimatedDish": true,  
  "dishCategory": "SPICED_CURRY", "rawText": "misal pav",  
  "grams": 300,  
  "caloriesKcal": 510, "kcalLow": 360, "kcalHigh": 660, // a RANGE, midpoint in caloriesKcal  
  "proteinG": null, "carbsG": null, "fatG": null, // genuinely unknown  
  "perGram": null }
```

RENDER

The calorie **range** (360–660 kcal) with the midpoint as the number that feeds the total. Macros show "—". Label it as an estimate.

DO NOT OFFER ADJUST PORTION

`PATCH` on an estimated-dish item returns `422 estimated_dish_not_editable`. Offer "Find this food" or "Break into ingredients" instead — that is the designed affordance. Gate the portion control on `isEstimatedDish === false`.

Also on the card

- **Meal name** — server-derived, editable. Persist a rename with `PATCH /assistant/drafts/{id}`.
- **Slot** — a segmented control. Same `PATCH` endpoint. See the slot gotcha in §11.
- **Confidence** — the fraction of items that resolved. Useful for deciding how prominent the "check this" nudge is; not something to show as a number.

7 Editing a draft — the tap-driven endpoints

Four calls. All of them take `version`, all of them return the full updated draft.

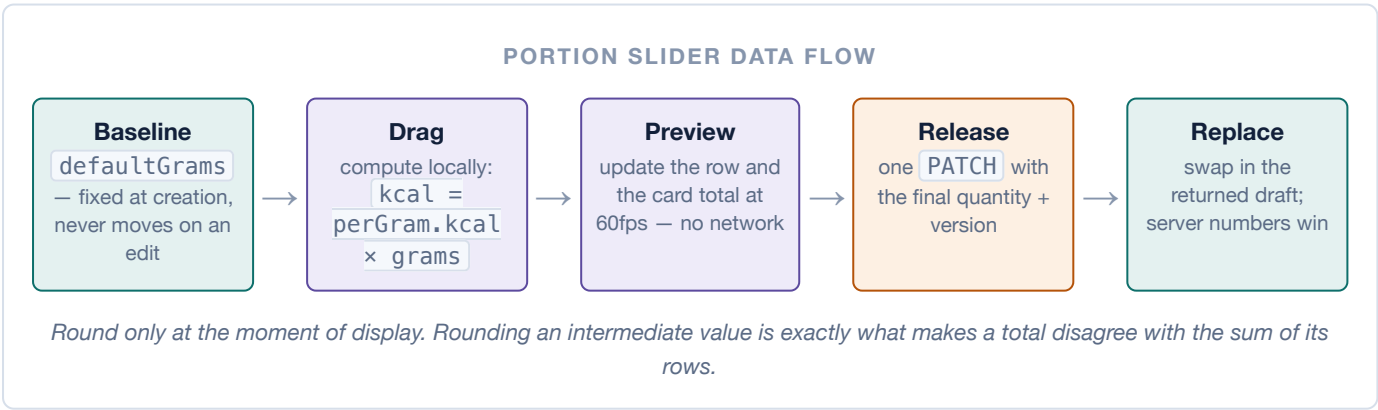
ACTION	CALL	BODY
Rename / change slot	PATCH <code>/assistant/drafts/{id}</code>	<code>{name?, slot?, version, opId?}</code>
Add an item (exact food)	POST <code>/assistant/drafts/{id}/items</code>	<code>{foodId, quantity, unit, state?, version, opId?}</code> → 201
Adjust portion	PATCH <code>.../items/{itemId}</code>	<code>{quantity?, unit?, state?, version, opId?}</code>
Remove an item	DELETE <code>.../items/{itemId}</code>	<code>{version, opId?}</code> — yes, a body on DELETE
Discard the draft	DELETE <code>/assistant/drafts/{id}</code>	<code>{version, opId?}</code>

Handling a version conflict

```
if (err.code === "draft_version_conflict") {  
  // the CURRENT draft is right here – no refetch needed  
  setDraft(err.details.draft);  
  toast("This meal changed on another device – showing the latest.");  
}
```

Adjust Portion — the one place you do client-side maths

The slider must feel instant, so the server hands you a per-gram rate and gets out of the way. `perGram` is deliberately unrounded — it is an input to your maths, not a display value.



```
// live preview while dragging – no network
const kcal    = item.perGram.kcal    * grams;
const protein = item.perGram.proteinG * grams;
// ...and the card total: recompute from all items at full precision, round once for display

// on release – persist exactly once
await patch(`/assistant/drafts/${draftId}/items/${item.id}`, {
  quantity: grams, unit: "g", version: draft.version
});
```

DO NOT PATCH PER FRAME

Every call bumps `version`, so a stream of them will make your own next call conflict with itself. Debounce to the release, or to a ~400 ms trailing edge.

8 Confirm & Log

```
POST /api/v1/assistant/drafts/{id}/confirm
```

```
{
  "idempotencyKey": "9cle...", // REQUIRED. Generate once per confirm attempt, reuse on retry
  "version": 3,                // REQUIRED

  // optional — see below
  "nutritionSnapshot": { "caloriesKcal": 498 },
  "mealTimestamp": "2026-09-06T12:30:00Z",
  "staleConfirmed": false
}
```

```
// 201 Created
{
  "loggedMeal": { "id", "name", "slot", "source": "CHAT_AI", "loggedAt", "totals", "items",
  "catalogVersionChanged" },
  "dailyTotals": { "caloriesKcal": 1240, "proteinG": 62, ... }, // today, across ALL meals
  "recomputedFromClientSnapshot": false
}
```

FIELD	WHY YOU CARE
<code>idempotencyKey</code>	The double-tap guard. Generate one UUID when the user taps Confirm and reuse it for every retry of that attempt. A repeat with the same key replays the original response instead of logging a second meal. Generate a <i>new</i> key only if the user backs out and confirms again.
<code>dailyTotals</code>	Update Home from this directly — no refetch. Note there is no <code>remainingKcal</code> ; compute it against the plan target from <code>GET /onboarding/plan</code> .
<code>nutritionSnapshot</code>	Optional. Send the calorie figure you last showed the user; if the server's fresh recomputation differs by more than the epsilon, <code>recomputedFromClientSnapshot</code> comes back <code>true</code> . Show a one-time "we updated this from the latest food data" note. It is never persisted.
<code>mealTimestamp</code>	Offline replay only. When the user actually ate, not when the request reached the server — so a backdated meal lands on the right day.
<code>staleConfirmed</code>	Set <code>true</code> only after the user explicitly confirms a >7-day-old queued entry (you get <code>422 operation_stale</code> first).

WHAT SILENTLY DOES NOT MAKE IT INTO THE MEAL

`UNRESOLVED` items never become logged items, and an estimated dish whose category profile has since been removed is dropped too. Surface unresolved items clearly *before* the confirm tap — the user should not discover the omission in their diary.

9 The wellbeing path

HANDLE THIS DELIBERATELY

When `intent === "WELLBEING_FLAG"`: `draft` is `null`, `gamificationSuppressed` is `true`, and `wellbeingResources` carries a list of support contacts.

- **Render `assistantText` plainly.** No celebration styling, no streak chrome, no progress rings anywhere on screen for this turn.
- **Show the resources with restraint.** The client decides whether and when to surface them — an unavoidable modal is the wrong call. A calm, dismissible card is closer.
- **Never block logging.** The user can keep using the app normally; do not gate the input.
- **Suppress gamification for the session** once `gamificationSuppressed` is true.

BEFORE YOU SHIP THIS

The reply copy and the resource list are unreviewed placeholders, and the resource list is US-only. Coordinate with whoever owns clinical / trust-and-safety review before this path is user-visible.

The quota pill

```
GET /api/v1/assistant/quota
→ { "used": 3, "limit": 20, "remaining": 17, "resetsAt": "2026-09-07T09:12:00Z" | null }
```

Read-only — checking never costs a unit. Fetch it when the chat screen opens, and again after any response whose `tier` is `LLM_SMALL` or `LLM_LARGE`. A `resetsAt` of `null` means the user has no active window (they have never used an AI parse, or the last one has expired). When `quotaExceeded` arrives, lead with what still works: quantified phrasing ("200g rice") and manual entry are both unaffected.

10 The manual path

Your fallback for every case the chat cannot handle — and the thing that keeps the app usable when quota runs out or the model is down.

TWO MANUAL ROUTES, AND WHEN TO USE EACH

Direct log

`POST /api/v1/meals` → a `LoggedMeal` immediately. No draft, no review step. Use for "add food" from the diary.

Structured draft

`POST /assistant/drafts` → a draft card the user reviews, then confirms. Use when you want the same review UI as chat.

```
GET /api/v1/meals/foods?q=rice
→ { "foods": [{ "id", "name", "source", "defaultState",
                  "caloriesKcalPer100g", "proteinGPer100g", "carbsGPer100g",
                  "fatGPer100g", "fiberGPer100g",
                  "servingUnits": [{ "unit": "katori", "grams": 150, "type": "HOUSEHOLD" }] } ] }
```

DRIVE THE UNIT PICKER FROM `SERVINGUNITS`

`g` and `kg` work for every food. Everything else — `cup`, `katori`, `piece` — is declared **per food**, because a cup of rice and a cup of spinach are not the same mass. Offer only that food's own units plus g/kg; sending anything else returns `422 invalid_quantity`.

```
POST /api/v1/meals // note: NO trailing slash
{ "name": "Lunch", "slot": "LUNCH",
  "items": [{ "foodId": "...", "quantity": 200, "unit": "g", "state": "COOKED" }] } // max 20 items
```

ALSO AVAILABLE

NOTES

`GET /api/v1/meals?slot=LUNCH`

Today's meals, optionally filtered. Drives the diary list.

`GET · DELETE /api/v1/meals/{id}`

Detail and delete. Delete returns 204.

`PATCH · DELETE`

`/api/v1/meals/{id}/items/{itemId}`

Edit a logged meal after the fact. Totals are recomputed server-side and returned.

`STATE` MATTERS MORE THAN IT LOOKS

Rice is catalogued cooked; 100 g stated `RAW` is treated as ~300 g cooked. If your UI lets the user pick raw vs cooked, send it. If you omit it, the food's own default basis is used — which is usually right, and always better than guessing.

Slot inference: `POST /assistant/drafts` accepts `localHour` (0–23, the device's local hour) and infers the slot from it when you omit `slot`. Send it — otherwise the server falls back to its own UTC clock.

11 Errors, retries and gotchas

Error code → what the UI should do

STATUS	CODE	RECOVERY
400	<code>validation_error</code>	A client bug. <code>details</code> is per-field. Log it; show a generic message.
401	—	On a guest token: it may have been claimed → switch to the Clerk session. Otherwise re-authenticate.
404	<code>draft_not_found</code>	Your stored <code>draftId</code> is stale. Clear it and start fresh.
404	<code>food_not_found</code> · <code>meal_not_found</code>	The catalog changed, or it is another user's id. Refresh the list.
409	<code>open_draft_exists</code>	Read <code>details.draft</code> and render it. Offer "continue" or "discard and start over".
409	<code>draft_version_conflict</code>	Read <code>details.draft</code> , replace local state, tell the user briefly. Do not refetch.
409	<code>draft_not_open</code>	Confirmed, discarded or expired. Show terminal state and clear the stored id.
409	<code>idempotency_key_reused</code>	You reused a key with different content. A client bug — fix the key generation.
422	<code>invalid_quantity</code>	Unrecognised unit for that food (<code>details.unit</code>). Restrict the picker to <code>servingUnits</code> + g/kg.
422	<code>estimated_dish_not_editable</code>	Hide portion controls when <code>isEstimatedDish</code> .
422	<code>operation_stale</code>	Offline entry older than 7 days. Ask the user to confirm, then resend with <code>staleConfirmed: true</code> .
422	<code>operation_expired</code>	Older than 30 days — cannot be logged. Surface it; do not silently drop it.
429	—	Only on <code>POST /auth/guest</code> . Back off.
503	—	Backend degraded. Retry with backoff; keep manual entry available if you have cached foods.

Retry safety — which calls take a key

CALL	KEY FIELD	NOTES
<code>POST /assistant/messages</code>	<code>clientId</code> required	24-hour replay window.
<code>POST .../confirm</code>	<code>idempotencyKey</code> required	31-day window — sized for the offline queue.
Draft create / add / patch / delete	<code>opId</code> <i>optional</i>	Omit it online. Send one for any request you might queue offline.
<code>POST /api/v1/meals</code>	none	Not idempotent. Guard double-taps in the UI yourself.

Gotchas worth writing on a sticky note

No trailing slashes <code>/api/v1/meals</code> and <code>/api/v1/assistant/drafts</code> are registered without one. Adding it triggers a redirect that most HTTP clients turn into a GET — your POST silently becomes a read.	DELETE carries a body Discarding a draft or removing an item needs <code>{version}</code> in the body. Some HTTP clients strip bodies from DELETE — check yours.
Chat drafts get a server-time slot <code>POST /messages</code> has no <code>localHour</code> field, so the slot is inferred from the server's UTC hour. For users far from UTC this will often be wrong. Either default the slot control to the device's own inference, or PATCH the draft right after creation.	Null is not zero Macros are nullable at item, draft, meal and daily-total level. Render "—". A <code>?? 0</code> anywhere in your formatters will produce confidently wrong numbers.
Totals are rounded once, by the server Don't sum the rounded per-item values and expect the card total — the server sums at full precision and rounds at the end. Trust <code>totals</code> .	"2 rotis" does not parse The grammar needs an explicit unit word: "2 pieces roti" works, "2 rotis" comes back in <code>unconsumedText</code> . Worth a hint in the input placeholder.

Client state — what to keep, what never to keep

KEEP	NEVER KEEP
The auth token · the open <code>draftId</code> · the chat transcript for display · in-flight idempotency keys until their call resolves	Draft nutrition across sessions (recomputed server-side) · a copy of the engine constants (fetch <code>GET /onboarding/config</code>) · anything derived from <code>perGram</code> beyond the current slider gesture

12 Worked examples — complete traces

Real request/response pairs against the seeded dev catalog. Every number is derivable from the catalog data, so you can use these as fixtures for your own tests.

THE CATALOG THESE EXAMPLES ASSUME

Cooked White Rice (130 kcal/100 g, cooked basis, 3.0 raw→cooked yield, units: katori 150 g · cup 158 g) · Whole Wheat Roti (297, piece 40 g) · Boiled Egg (155, piece 50 g) · Grilled Chicken Breast (165) · Olive Oil (884, tbsp 13.5 g) · Banana (89, piece 118 g) · Spinach (23, cup 30 g) · Toor Dal (—). Composites: Chicken Biryani (350 g plate) · Dal Chawal · Roti Sabzi. Run `manage.py seed_foods && seed_composite_foods && seed_dish_category_profiles` to get them.

EX 1 The happy path — free text to a logged meal

4 calls

STEP 1 — SEND THE MESSAGE

```
POST /api/v1/assistant/messages
{ "clientId": "a1b2c3d4-0001", "content": "200g rice and 2 pieces roti" }
```

200 OK

```
{
  "requestId": "7f3e...", "messageId": "msg_01",
  "tier": "PARSER", "intent": "LOG_NEW",
  "assistantText": "Got it — let me break that down.",
  "draft": {
    "id": "drf_01", "version": 1, "status": "OPEN",
    "name": "Lunch — Whole Wheat Roti", "slot": "LUNCH",
    "parseTier": "PARSER", "confidence": 1.0,
    "totals": { "caloriesKcal": 498, "proteinG": 14, "carbsG": 101, "fatG": 4, "fiberG": 7 },
    "items": [
      { "id": "itm_01", "resolution": "RESOLVED", "isEstimatedDish": false,
        "foodId": "fd_rice", "foodName": "Cooked White Rice", "rawText": "200g rice",
        "quantity": 200, "unit": "g", "grams": 200, "state": "COOKED", "defaultGrams": 200,
        "quantitySource": "EXPLICIT", "massSource": "DIRECT", "matchBand": "HIGH",
        "caloriesKcal": 260, "proteinG": 5, "carbsG": 56, "fatG": 1, "fiberG": 1,
        "perGram": { "kcal": 1.3, "proteinG": 0.027, "carbsG": 0.282, "fatG": 0.003 } },
      { "id": "itm_02", "resolution": "RESOLVED", "isEstimatedDish": false,
        "foodId": "fd_roti", "foodName": "Whole Wheat Roti", "rawText": "2 pieces roti",
        "quantity": 2, "unit": "piece", "grams": 80, "state": "COOKED", "defaultGrams": 80,
        "quantitySource": "EXPLICIT", "massSource": "HOUSEHOLD_TABLE", "matchBand": "HIGH",
        "caloriesKcal": 238, "proteinG": 9, "carbsG": 45, "fatG": 4, "fiberG": 6,
        "perGram": { "kcal": 2.97, "proteinG": 0.11, "carbsG": 0.56, "fatG": 0.046 } }
    ]
  },
  "unconsumedText": [], "needsClarification": null,
  "quotaExceeded": false, "gamificationSuppressed": false, "wellbeingResources": null
}
```

RENDER

Assistant bubble, then a draft card titled *Lunch — Whole Wheat Roti* with two rows and a **498 kcal** total. Both items are HIGH confidence and EXPLICIT, so no "est." chips. Store `drf_01` and `version: 1`.

STEP 2 — USER DRAGS THE RICE TO 250 G AND RELEASES

```
PATCH /api/v1/assistant/drafts/drf_01/items/itm_01
{ "quantity": 250, "unit": "g", "version": 1 }
```

200 OK — THE FULL DRAFT COMES BACK

```
{ "id": "drf_01", "version": 2, "status": "OPEN", ...
  "totals": { "caloriesKcal": 563, "proteinG": 16, "carbsG": 115, "fatG": 4, "fiberG": 7 },
  "items": [
    { "id": "itm_01", ..., "grams": 250,
      "defaultGrams": 200,           // ← unchanged: the slider baseline never moves
      "caloriesKcal": 325, "proteinG": 7, "carbsG": 71, "fatG": 1, "fiberG": 1 },
    { "id": "itm_02", ... }
  ] }
```

NOTE

While dragging you showed $1.3 \times 250 = 325$ locally with no network. The PATCH fires once, on release, and the returned draft replaces your local copy. **version** is now **2** — carry it on the next call.

STEP 3 — CONFIRM & LOG

```
POST /api/v1/assistant/drafts/drf_01/confirm
{ "idempotencyKey": "cf-9c1e-0001", "version": 2,
  "nutritionSnapshot": { "caloriesKcal": 563 } } // optional: what you showed the user
```

201 CREATED

```
{
  "loggedMeal": {
    "id": "ml_01", "name": "Lunch — Whole Wheat Roti", "slot": "LUNCH",
    "source": "CHAT_AI", "loggedAt": "2026-09-06T12:41:07Z",
    "totals": { "caloriesKcal": 563, "proteinG": 16, "carbsG": 115, "fatG": 4, "fiberG": 7 },
    "items": [ ... ], "catalogVersionChanged": false
  },
  "dailyTotals": { "caloriesKcal": 1063, "proteinG": 48, "carbsG": 180, "fatG": 22, "fiberG": 14 },
  "recomputedFromClientSnapshot": false
}
```

RENDER

Dismiss the draft card, push the meal into the diary, and update Home straight from `dailyTotals` — no refetch.

`recomputedFromClientSnapshot: false` means the server agreed with your 563, so say nothing. **Clear the stored `draftId`.**

STEP 4 — THE USER DOUBLE-TAPPED; THE SAME REQUEST FIRES AGAIN

```
POST /api/v1/assistant/drafts/drf_01/confirm
{ "idempotencyKey": "cf-9c1e-0001", "version": 2 } // SAME key
```

201 — BYTE-IDENTICAL REPLAY, "ID": "ML_01"

WHY THIS MATTERS

No second meal was created. This only works because you reused the key. Generate a new one only if the user backs out and confirms again.

EX 2 A composite dish expands into its components

REQUEST

```
POST /api/v1/assistant/messages
{ "clientId": "...-0002", "content": "1 plate chicken biryani" }
```

200 OK — ONE PHRASE, THREE ITEMS

```
{ "tier": "PARSER", "intent": "LOG_NEW",
  "assistantText": "Got it — let me break that down.",
  "draft": { "id": "drf_02", "version": 1, "name": "Lunch — Cooked White Rice", "slot": "LUNCH",
    "totals": { "caloriesKcal": 624, "proteinG": 39, "carbsG": 64, "fatG": 22, "fiberG": 1 },
    "items": [
      { "id": "i1", "resolution": "RESOLVED", "foodName": "Cooked White Rice",
        "rawText": "Cooked White Rice (Chicken Biryani)", // ← names the parent dish
        "quantity": 227.5, "unit": "g", "grams": 227.5,
        "matchBand": null, // ← the DISH matched, not this row
        "caloriesKcal": 296, "proteinG": 6, "carbsG": 64, "fatG": 1, "fiberG": 1 },
      { "id": "i2", "resolution": "RESOLVED", "foodName": "Grilled Chicken Breast",
        "rawText": "Grilled Chicken Breast (Chicken Biryani)",
        "grams": 105, "matchBand": null,
        "caloriesKcal": 173, "proteinG": 33, "carbsG": 0, "fatG": 4, "fiberG": 0 },
      { "id": "i3", "resolution": "RESOLVED", "foodName": "Olive Oil",
        "rawText": "Olive Oil (Chicken Biryani)",
        "grams": 17.5, "matchBand": null,
        "caloriesKcal": 155, "proteinG": 0, "carbsG": 0, "fatG": 18, "fiberG": 0 } ] } }
```

RENDER

350 g plate × 0.65 / 0.30 / 0.05. Consider grouping the three rows under a "Chicken Biryani" header using the parenthesised part of `rawText` — three unexplained rows for one thing the user said reads as a bug. Each row is independently editable, which is the point of expanding at all.

Also try: `"1 plate biryani"` (a curated alias) gives an identical result. `"200g dal"` resolves to the standalone Toor Dal and does **not** expand into Dal Chawal — composites match exactly, never on a fragment.

EX 3 Descriptive input — the AI tier and an assumed quantity

needs a backend API key

REQUEST

```
POST /api/v1/assistant/messages
{ "clientId": "...-0003", "content": "I had a couple of boiled eggs and some rice for breakfast"
}
```

200 OK

```
{ "tier": "LLM_SMALL",           // ← a paid call happened; refresh the quota pill after this
  "intent": "LOG_NEW",
  "draft": { "id": "drf_03", "version": 1,
    "slot": "BREAKFAST",           // ← stated explicitly, so the model reported it
    "confidence": 1.0,
    "totals": { "caloriesKcal": 350, "proteinG": 17, "carbsG": 43, "fatG": 11, "fiberG": 1 },
    "items": [
      { "foodName": "Boiled Egg", "rawText": "boiled eggs",
        "quantity": 2, "unit": "piece", "grams": 100,
        "quantitySource": "EXPLICIT", "massSource": "HOUSEHOLD_TABLE",
        "caloriesKcal": 155, "proteinG": 13, "carbsG": 1, "fatG": 11 },
      { "foodName": "Cooked White Rice", "rawText": "rice",
        "quantity": 150, "unit": "g", "grams": 150,
        "quantitySource": "ASSUMED",           // ← THE USER NEVER SAID AN AMOUNT
        "massSource": "CATEGORY_FALLBACK",     // ← grain default: 150 g
        "caloriesKcal": 195, "proteinG": 4, "carbsG": 42, "fatG": 0 } ] } }
```

RENDER

The rice row **must** carry an "est." chip — `quantitySource: "ASSUMED"` means the server filled in a number the user never gave. Make that row's portion control the most obvious thing on the card. Once this user has logged rice explicitly three times, `massSource` becomes `"USER_HISTORY"` and the number becomes their own median — a good moment for a quiet *"your usual"* label.

EX 4 A food the catalog does not have

REQUEST

```
POST /api/v1/assistant/messages
{ "clientMessageId": "...-0004", "content": "150g quinoa and a bit of leftover curry" }
```

200 OK

```
{ "tier": "PARSER", "intent": "LOG_NEW",
  "draft": { "id": "drf_04", "version": 1,
    "confidence": 0.0,
    "totals": { "caloriesKcal": 0, "proteinG": null, "carbsG": null, "fatG": null, "fiberG": null },
    "items": [
      { "id": "i1", "resolution": "UNRESOLVED",
        "rawText": "150g quinoa",
        "foodId": null, "foodName": null,
        "quantity": null, "unit": null, "grams": null,
        "caloriesKcal": null, "proteinG": null, "perGram": null } ] },
  "unconsumedText": ["a bit of leftover curry"] }
```

RENDER

Two *different* failures, and the user needs to see both. **The item** — a row showing "150g quinoa", no numbers, and a **Search for it** CTA. **The unconsumed text** — a quiet line under the card: *"I didn't catch: a bit of leftover curry."*

Note `totals.proteinG` is `null`, not 0 — render "—". And this draft would confirm to an **empty meal**, because unresolved items are dropped: either disable Confirm or warn plainly.

EX 5 An estimated dish — a range, and no macros

needs a backend API key

REQUEST

```
POST /api/v1/assistant/messages
{ "clientId": "...-0005", "content": "I had a plate of misal pav" }
```

200 OK

```
{ "tier": "LLM_SMALL", "intent": "LOG_NEW",
  "draft": { "id": "drf_05", "version": 1,
    "totals": { "caloriesKcal": 510,
      "proteinG": null, "carbsG": null, "fatG": null, "fiberG": null },
    "items": [
      { "id": "i1", "resolution": "ESTIMATED_DISH", "isEstimatedDish": true,
        "dishCategory": "SPICED_CURRY", "rawText": "misal pav",
        "foodId": null, "foodName": null,
        "quantity": 300, "unit": "g", "grams": 300,
        "caloriesKcal": 510, // the midpoint — this is what feeds the total
        "kcalLow": 360, "kcalHigh": 660, // the curated p25–p75 band × 300 g
        "proteinG": null, "carbsG": null, "fatG": null, "fiberG": null,
        "perGram": null } ] } }
```

RENDER

Misal Pav · ~510 kcal (360–660) with an "estimate" label, macros as "—", and **no portion slider** (`perGram` is null and PATCH returns 422). Offer *Find this food* and *Break into ingredients* instead. The dish name has been filed to the curation backlog automatically, so this improves on its own over time.

EX 6 Conversational edits on an open draft

Continuing from Ex 1, with `drf_01` open at version 1.

THREE MESSAGES IN SEQUENCE

```
{ "clientId": "...-0006", "content": "make the rice 250g" }
{ "clientId": "...-0007", "content": "that was dinner" }
{ "clientId": "...-0008", "content": "remove the roti" }
```

THREE RESPONSES, ABRIDGED

```
→ { "tier": "PARSER", "intent": "EDIT_ITEM", "assistantText": "Updated.",
    "draft": { "version": 2, "totals": { "caloriesKcal": 563, ... } } }

→ { "tier": "PARSER", "intent": "SET_SLOT", "assistantText": "Moved to dinner.",
    "draft": { "version": 3, "slot": "DINNER" } }

→ { "tier": "PARSER", "intent": "REMOVE_ITEM", "assistantText": "Removed.",
    "draft": { "version": 4, "totals": { "caloriesKcal": 325, ... },
      "items": [ { "id": "itm_01", ... } ] } }
```

RENDER

Each response returns the whole draft — replace, don't patch locally. Animating the changed row makes the connection between what the user typed and what moved. All three are `PARSER`: free, instant, no quota. Note the user reached the same outcomes as the tap endpoints in §7 — support both, they converge.

EX 7 A new meal while a draft is open — the clarification round trip

REQUEST — DRAFT `DRF_01` IS STILL OPEN

```
POST /api/v1/assistant/messages
{ "clientId": "...-0009", "content": "300g rice" }
```

200 OK — NOTHING WAS MUTATED

```
{ "tier": "PARSER", "intent": "LOG_NEW",
  "assistantText": "You already have a meal in progress — add this to it, or start a new one?",
  "draft": { "id": "drf_01", "version": 4, ... }, // ← the EXISTING draft, unchanged
  "needsClarification": { "reason": "open_draft", "candidates": ["ADD", "NEW"] } }
```

USER TAPS "ADD TO THIS MEAL" — RESEND WITH A NEW CLIENTMESSAGEID

```
POST /api/v1/assistant/messages
{ "clientId": "...-0010",           // ← NEW id: this is a different request
  "content": "300g rice",           // ← same content
  "onOpenDraft": "ADD" }
```

200 OK — MERGED

```
{ "tier": "PARSER", "intent": "LOG_NEW", "assistantText": "Added.",
  "draft": { "id": "drf_01", "version": 5, "items": [ ..., { "foodName": "Cooked White Rice", ... } ] },
  "needsClarification": null }
```

RENDER

Two chips under the assistant bubble. Because `draft` came back non-null, keep showing the existing card — it is context for the question, not a change. Choosing `"NEW"` instead discards `drf_01` and creates a fresh draft, so word that chip carefully: *"Start a new meal (discards the current one)"*.

EX 8 Non-logging intents — four one-liners

```
// "hi"
→ { "tier": "PRECLASSIFIER", "intent": "SOCIAL", "draft": null,
    "assistantText": "Hey 🙌 What did you eat?" }

// "how many calories left" ← a real database read, not canned copy
→ { "tier": "PRECLASSIFIER", "intent": "DIARY_QUERY", "draft": null,
    "assistantText": "So far today: 1,063 kcal – 797 kcal left toward your 1,860 kcal goal." }

// "how much protein is in an egg"
→ { "tier": "PRECLASSIFIER", "intent": "NUTRITION_QA", "draft": null,
    "assistantText": "Boiled Egg (per 100g): 155 kcal, 13g protein, 1g carbs, 11g fat." }

// "should i try keto"
→ { "tier": "PRECLASSIFIER", "intent": "ADVICE_SEEKING", "draft": null,
    "assistantText": "I can't give dietary or medical advice — that's best discussed with a
                    registered dietitian or your doctor. I can help you log what you eat though!" }
```

RENDER

All four: assistant bubble only, no card, no quota consumed. Two deserve extra work — make the **DIARY_QUERY** numbers tappable through to the diary, and if the answer mentions Insights (it does for "this week" style questions), deep-link it. For **NUTRITION_QA**, link the food name to its detail screen.

EX 9 Quota exhausted — still HTTP 200

REQUEST — THE USER HAS USED ALL 20 AI PARSES TODAY

```
POST /api/v1/assistant/messages
{ "clientId": "...-0011", "content": "I ate some rice with a bit of chicken" }
```

200 OK — NOTE THE STATUS

```
{ "tier": "PARSER", "intent": "OTHER", "draft": null,
  "assistantText": "Quantified meals still work — try \"200g rice, 100g chicken\".",
  "quotaExceeded": true }
```

FOLLOW UP FOR THE NUMBERS

```
GET /api/v1/assistant/quota
→ { "used": 20, "limit": 20, "remaining": 0, "resetsAt": "2026-09-07T09:12:00Z" }
```

RENDER

Not an error state — **lead with what still works**. Show the assistant's suggestion prominently, a "resets in 8 h" pill, and a shortcut to manual entry. A quantified message like `"200g rice"` works right now, unaffected, because it never reaches the quota gate.

EX 10 A version conflict, and recovering from the error body

REQUEST — YOU HAVE VERSION 2, BUT A SECOND DEVICE ALREADY MOVED IT TO 3

```
PATCH /api/v1/assistant/drafts/drf_01/items/itm_01
{ "quantity": 250, "unit": "g", "version": 2 }
```

409 CONFLICT — WITH EVERYTHING YOU NEED TO RECOVER

```
{ "error": {
  "code": "draft_version_conflict",
  "message": "This draft has changed since you last read it.",
  "details": { "draft": { // ← the CURRENT draft, in full
    "id": "drf_01", "version": 3, "status": "OPEN",
    "totals": { "caloriesKcal": 401, ... },
    "items": [ ... ] } },
  "requestId": "b21c..." } }
```

HANDLE

`setDraft(err.details.draft)`, then a brief toast. **Do not refetch** — the fresh state is already in your hands. Then decide whether to replay the user's intent against version 3 automatically (fine for a portion change) or ask them to redo it (safer for a delete). `open_draft_exists` and `draft_not_open` carry the same `details.draft` payload, so one recovery helper covers all three.

EX 11 The manual path — search, then log directly

SEARCH

```
GET /api/v1/meals/foods?q=rice
```

200 OK

```
{ "foods": [
  { "id": "fd_rice", "name": "Cooked White Rice", "source": "CALORYX_CURATED",
    "defaultState": "COOKED",
    "caloriesKcalPer100g": 130, "proteinGPer100g": 2.7,
    "carbsGPer100g": 28.2, "fatGPer100g": 0.3, "fiberGPer100g": 0.4,
    "servingUnits": [ { "unit": "katori", "grams": 150, "type": "HOUSEHOLD" },
                      { "unit": "cup", "grams": 158, "type": "HOUSEHOLD" } ] } ] }
```

BUILD THE UNIT PICKER FROM THIS

Offer `g`, `kg`, `katori`, `cup` — and nothing else. Sending `"bowl"` for this food returns `422 invalid_quantity` with `details: {"unit":"bowl"}`.

LOG IT — NO DRAFT, NO REVIEW STEP

```
POST /api/v1/meals // no trailing slash
{ "name": "Lunch", "slot": "LUNCH",
  "items": [ { "foodId": "fd_rice", "quantity": 1, "unit": "katori" } ] }
```

201 CREATED

```
{ "id": "ml_02", "name": "Lunch", "slot": "LUNCH", "source": "MANUAL",
  "loggedAt": "2026-09-06T13:02:11Z",
  "totals": { "caloriesKcal": 195, "proteinG": 4, "carbsG": 42, "fatG": 0, "fiberG": 1 },
  "items": [ { "id": "...", "foodName": "Cooked White Rice",
               "quantity": 1, "unit": "katori", "grams": 150, "state": "COOKED",
               "caloriesKcal": 195, "proteinG": 4, "carbsG": 42, "fatG": 0, "fiberG": 1 } ],
  "catalogVersionChanged": false }
```

WATCH OUT

This endpoint has **no idempotency key** — guard the double-tap in your own UI (disable the button while in flight). Also note 1 katori resolved to 150 g server-side; you never do that conversion yourself.

EX 12 Raw vs cooked — the one that silently triples a number

```
// The SAME 100 g, logged two different ways

{ "items": [ { "foodId": "fd_rice", "quantity": 100, "unit": "g" } ] }
→ "grams": 100, "state": "COOKED", "caloriesKcal": 130

{ "items": [ { "foodId": "fd_rice", "quantity": 100, "unit": "g", "state": "RAW" } ] }
→ "grams": 100, "state": "RAW", "caloriesKcal": 390 // ← 3× the yield factor
```

WHY

Rice is catalogued on a **cooked** basis, so 100 g of *raw* rice becomes ~300 g cooked before the lookup. `grams` still reports what the user stated — display that, not the converted mass.

If your UI exposes a raw/cooked toggle, send `state`. If it doesn't, omit it and the food's own default applies — which is right far more often than a guess. A food with no yield factor (Whole Wheat Roti) returns `422 invalid_quantity` if you send a mismatched state, so only show the toggle when the food actually has a basis worth switching.

13 Code you can copy

Framework-agnostic TypeScript. Adapt the storage and state calls to whatever you are using.

The API client — parse the error envelope once

```
export class ApiError extends Error {
  constructor(
    readonly status: number,
    readonly code: string,
    message: string,
    readonly details: Record<string, any> = {},
    readonly requestId?: string,
  ) { super(message); }
}

export async function api<T>(path: string, init: RequestInit = {}): Promise<T> {
  const requestId = crypto.randomUUID();
  const res = await fetch(`${BASE_URL}/api/v1${path}`, {
    ...init,
    headers: {
      "Content-Type": "application/json",
      "X-Request-Id": requestId,
      Authorization: `Bearer ${await getToken()}`,
      ...init.headers,
    },
  });

  if (res.status === 204) return undefined as T;

  const body = await res.json();
  if (!res.ok) {
    const e = body?.error ?? {};
    // a guest token that was claimed also 401s — see §2
    if (res.status === 401) await onUnauthenticated();
    throw new ApiError(res.status, e.code ?? "unknown", e.message ?? "Request failed",
      e.details ?? {}, e.requestId ?? requestId);
  }
  return body as T;
}
```

Sending a message — the whole branch, in one function

```
type SendResult =
  | { kind: "clarify"; text: string; reason: string; candidates: string[]; draft: Draft | null }
  | { kind: "draft"; text: string; draft: Draft; unconsumed: string[] }
  | { kind: "reply"; text: string; intent: ChatIntent };

export async function sendMessage(
  content: string,
  opts: { clientId?: string; onOpenDraft?: "ADD" | "NEW" } = {},
): Promise<SendResult & { quotaExceeded: boolean; wellbeing: boolean }> {

  // generate ONCE per message; reuse the same id when retrying THIS message
  const clientId = opts.clientId ?? crypto.randomUUID();

  const r = await api<MessageResponse>("/assistant/messages", {
    method: "POST",
    body: JSON.stringify({ clientId, content, onOpenDraft: opts.onOpenDraft }),
  });

  const flags = {
    quotaExceeded: r.quotaExceeded,
    wellbeing: r.intent === "WELLBEING_FLAG",
  };

  // order matters: clarification first — NOTHING was mutated in that case
  if (r.needsClarification) {
    return { kind: "clarify", text: r.assistantText, draft: r.draft,
      reason: r.needsClarification.reason,
      candidates: r.needsClarification.candidates, ...flags };
  }
  if (r.draft) {
    return { kind: "draft", text: r.assistantText, draft: r.draft,
      unconsumed: r.unconsumedText, ...flags };
  }
  return { kind: "reply", text: r.assistantText, intent: r.intent, ...flags };
}
```

RETRYING

On a network failure, call `sendMessage` again with the **same** `clientId`. Answering a clarification is a *different* request — pass no id so a fresh one is generated.

Rendering an item — one switch, three shapes

```
function ItemRow({ item }: { item: DraftItem }) {
  switch (item.resolution) {

    case "UNRESOLVED":
      return <Row
        title={item.rawText}
        subtitle="We couldn't find this food"
        value="-"
        action={{ label: "Search for it", onPress: () => openSearch(item.rawText) }}
      />;

    case "ESTIMATED_DISH":
      return <Row
        title={item.rawText}
        subtitle={`Estimate · ${item.kcalLow}-${item.kcalHigh} kcal`}
        value={`~${item.caloriesKcal} kcal`}
        macros="-" // null, not zero
        editable={false} // PATCH would 422
        action={{ label: "Find this food", onPress: () => openSearch(item.rawText) }}
      />;

    default: // RESOLVED
      return <Row
        title={item.foodName!}
        subtitle={formatAmount(item)}
        badge={item.quantitySource === "ASSUMED" ? "est." : undefined}
        hint={item.massSource === "USER_HISTORY" ? "your usual" : undefined}
        value={` ${item.caloriesKcal} kcal`}
        macros={fmtMacros(item)}
        editable
      />;
  }
}

// null means UNKNOWN. Never `?? 0`.
const g = (v: number | null) => (v === null ? "-" : `${v}g`);
const fmtMacros = (i: DraftItem) =>
  `P ${g(i.proteinG)} · C ${g(i.carbsG)} · F ${g(i.fatG)}`;
```

The portion slider — local maths, one write

```
function usePortion(draft: Draft, item: DraftItem) {
  const [grams, setGrams] = useState(item.grams ?? 0);
  const [saving, setSaving] = useState(false);

  // live preview — no network, full precision, round only here for display
  const preview = item.perGram && {
    kcal: Math.round(item.perGram.kcal * grams),
    proteinG: Math.round(item.perGram.proteinG * grams),
    carbsG: Math.round(item.perGram.carbsG * grams),
    fatG: Math.round(item.perGram.fatG * grams),
  };

  // persist exactly once, on release — never per frame
  const commit = async () => {
    setSaving(true);
    try {
      const updated = await api<Draft>({
        path: `/assistant/drafts/${draft.id}/items/${item.id}`,
        method: "PATCH",
        body: JSON.stringify({ quantity: grams, unit: "g", version: draft.version }) },
      );
      setDraft(updated); // replace wholesale; server numbers win
    } catch (e) {
      if (e instanceof ApiError) recoverFromDraftError(e);
    } finally { setSaving(false); }
  };

  return { grams, setGrams, preview, commit, saving,
    baseline: item.defaultGrams }; // the slider's origin — never moves
}
```

One recovery helper for all three draft conflicts

```
export function recoverFromDraftError(e: ApiError): boolean {
  switch (e.code) {
    case "draft_version_conflict":
      setDraft(e.details.draft); // fresh state is IN the error
      toast("This meal changed on another device – showing the latest.");
      return true;

    case "open_draft_exists":
      setDraft(e.details.draft); // show what they already have
      promptContinueOrDiscard();
      return true;

    case "draft_not_open":
      clearStoredDraftId(); // confirmed / discarded / expired
      setDraft(null);
      toast("That meal was already finished.");
      return true;

    case "draft_not_found":
      clearStoredDraftId();
      setDraft(null);
      return true;

    default:
      return false; // not ours – let it bubble
  }
}
```

Confirm — one key per attempt

```
export async function confirmDraft(draft: Draft, shownKcal: number) {
  // generated ONCE for this attempt and reused by the retry wrapper
  const idempotencyKey = crypto.randomUUID();

  const res = await withRetry(() =>
    api<ConfirmResponse>(`/assistant/drafts/${draft.id}/confirm`, {
      method: "POST",
      body: JSON.stringify({
        idempotencyKey,
        version: draft.version,
        nutritionSnapshot: { caloriesKcal: shownKcal }, // optional drift check
      }),
    }),
  );

  applyDailyTotals(res.dailyTotals); // update Home without a refetch
  addMealToDiary(res.loggedMeal);
  clearStoredDraftId();
  setDraft(null);

  if (res.recomputedFromClientSnapshot) {
    toast("We updated this meal from the latest food data.");
  }
  return res.loggedMeal;
}
```

Finding the open draft on launch

```
// There is no GET /assistant/drafts (see §3). Persist the id, and fall back
// to the 409 conflict body when local storage is empty or stale.
export async function restoreOpenDraft(): Promise<Draft | null> {
  const id = await storage.get("openDraftId");
  if (!id) return null;
  try {
    const draft = await api<Draft>(`/assistant/drafts/${id}`);
    // a stale draft flips to EXPIRED on this very read
    if (draft.status !== "OPEN") { await storage.del("openDraftId"); return null; }
    return draft;
  } catch (e) {
    if (e instanceof ApiError && e.status === 404) await storage.del("openDraftId");
    return null;
  }
}

// ...and whenever a response gives you a draft:
function setDraft(d: Draft | null) {
  draftStore.set(d);
  if (d?.status === "OPEN") storage.set("openDraftId", d.id);
  else storage.del("openDraftId");
}
```

14 Suggested build order

Each stage is independently demoable and unblocks the next. Stages 1–4 need no AI key on the backend at all.

BUILD ORDER

1

Auth + resume
POST /auth/guest, GET /auth/session, token storage, the 401-on-claimed-guest branch. Everything else needs this.

2

Manual logging
Food search, unit picker from servingUnits, POST /meals, the diary list. No AI, no drafts — proves your nutrition rendering before any of the hard parts.

3

Chat send + draft card
POST /messages, render assistantText and the draft, all three item states. Test against T1 phrasing ("200g rice and 2 pieces roti") which works with no AI key.

4

Editing + confirm
Portion slider on perGram, add/remove, version-conflict recovery, confirm with an idempotency key, Home totals from dailyTotals.

5

The branches
Both clarifications, unconsumedText, quota pill and the exceeded sheet, non-logging intents with their deep links.

6

Wellbeing + estimated dishes
Needs the backend AI key to exercise. Coordinate the wellbeing copy review before it ships.

7

Offline queue
opId on queued mutations, mealTimestamp on confirm, the stale/expired flows. Do this last — it is the only part with no server-side counterpart to lean on.

Definition of done for the chat screen

CHECK	
☐	Every message carries a fresh clientMessageId; a retry reuses it.
☐	needsClarification is checked before draft, and renders chips for both reasons.
☐	All three item states render distinctly, and estimated dishes have no portion control.
☐	Null macros render as "—" everywhere, including totals and daily totals.
☐	unconsumedText is always surfaced.
☐	quantitySource: "ASSUMED" shows an "est." affordance.

CHECK

- ☐ Version conflicts recover from `error.details.draft` without a refetch.
- ☐ Confirm uses one idempotency key per attempt; double-tapping creates one meal.
- ☐ Quota exceeded leads with what still works, not with the limit.
- ☐ The open draft survives an app restart (stored id, or recovery from 409 `open_draft_exists`).
- ☐ `WELLBEING_FLAG` suppresses all gamification chrome for the session.

TWO THINGS TO RAISE WITH BACKEND BEFORE YOU START

1. No list-drafts endpoint (§3) — decide whether resume-after-app-kill is a launch requirement, and if so ask for `GET /assistant/drafts`. **2. Chat slot inference uses server UTC** (§11) — either accept the PATCH-after-create workaround or ask for a `localHour` field on `POST /messages`.